

Aufgabe 11: Metrische Kovariablen

Der auf der Moodle-Homepage verfügbare Datensatz `wagen.txt` enthält Informationen zu 172 VW Golf-Modellen. Die folgende Tabelle gibt einen Überblick über die Variablen des Datensatzes:

Variable	Beschreibung
<code>preis</code>	Verkaufspreis [in 1000 €]
<code>alter</code>	Alter des Autos [in Monaten]
<code>kilstand</code>	Kilometerstand [in 1000 km]
<code>tuev</code>	Anzahl der Monate bis zur nächsten TÜV-Untersuchung
<code>sonderaus1</code>	Sonderaustattung vorhanden? (Ja = 1, Nein = 0)

In dieser Aufgabe soll der Verkaufspreis eines Autos in Abhängigkeit des Alters und des Kilometerstandes modelliert werden.

- (a) Lesen Sie den Datensatz in **R** und visualisieren Sie das Alter und den Verkaufspreis sowie den Kilometerstand und den Verkaufspreis jeweils in einem Streudiagramm. Was fällt bezüglich der beiden dargestellten Zusammenhänge auf?

Lösung:

- Einlesen der Daten und Überblick über Datensatz:

```
wagen <- read.csv("../Daten/wagen.txt", header = TRUE, sep = ",")
head(wagen)

##   preis alter kilstand tuev sonderaus1 sonderaus2
## 1  7.30   73      10   12           1           1
## 2  3.85  115      30   20           1           0
## 3  2.95  127      43    6           0           1
## 4  4.80  104      54   25           1           1
## 5  6.20   86      57   23           0           0
## 6  5.90   74      57   25           1           0

str(wagen)

## 'data.frame': 172 obs. of  6 variables:
##  $ preis      : num  7.3 3.85 2.95 4.8 6.2 5.9 4.9 4.7 6.35 5.9 ...
##  $ alter      : int  73 115 127 104 86 74 108 110 90 78 ...
##  $ kilstand   : num  10 30 43 54 57 57 58 65 67.5 68 ...
##  $ tuev       : int  12 20 6 25 23 25 25 19 19 12 ...
##  $ sonderaus1: int  1 1 0 1 0 1 1 1 1 0 ...
##  $ sonderaus2: int  1 0 1 1 0 0 1 1 1 0 ...

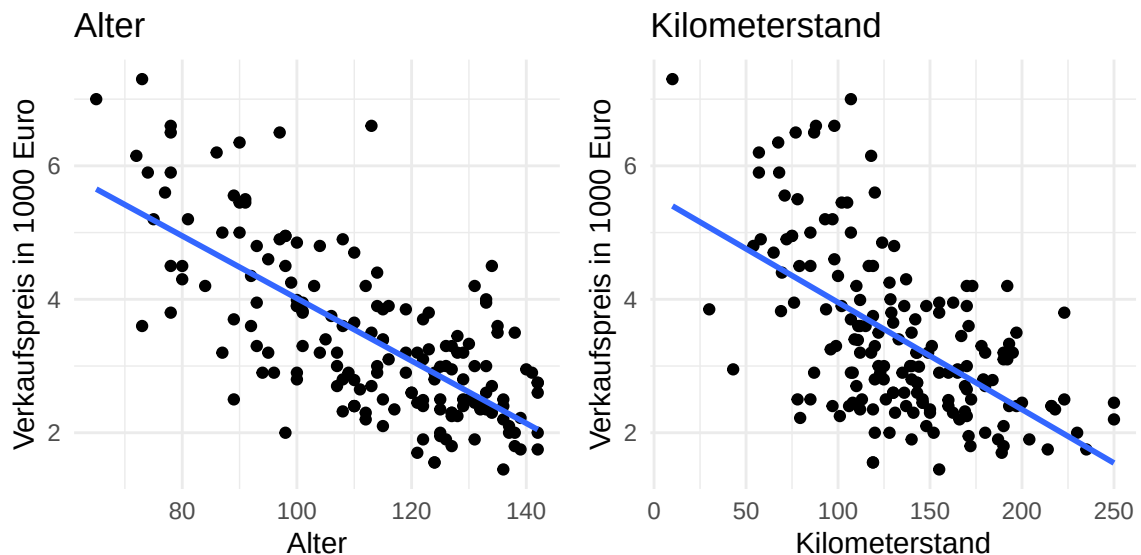
summary(wagen)

##      preis      alter      kilstand      tuev
##  Min.   :1.450   Min.   : 65.00   Min.   : 10.0   Min.   : 0.00
##  1st Qu.:2.450   1st Qu.: 99.75   1st Qu.:107.0   1st Qu.:11.00
##  Median :3.100   Median :115.00   Median :130.8   Median :19.00
##  Mean   :3.397   Mean   :113.19   Mean   :134.7   Mean   :16.98
##  3rd Qu.:3.960   3rd Qu.:129.00   3rd Qu.:167.5   3rd Qu.:24.00
##  Max.   :7.300   Max.   :142.00   Max.   :250.0   Max.   :26.00
```

```
##      sonderaus1      sonderaus2
## Min.      :0.000   Min.      :0.00
## 1st Qu.:0.000   1st Qu.:0.75
## Median :1.000   Median :1.00
## Mean      :0.686   Mean      :0.75
## 3rd Qu.:1.000   3rd Qu.:1.00
## Max.      :1.000   Max.      :1.00
```

- Graphische Visualisierung von Kovariablen und Zielgröße in Streudiagrammen:

```
# Alter:
plot1 <- ggplot(data = wagen, mapping = aes(x = alter, y = preis)) +
  geom_point() + stat_smooth(method = "lm", formula = y ~ x, se = FALSE) +
  theme_minimal() + ggtitle("Alter") + xlab("Alter") +
  ylab("Verkaufspreis in 1000 Euro")
# Kilometerstand:
plot2 <- ggplot(data = wagen, mapping = aes(x = kilstand, y = preis)) +
  geom_point() + stat_smooth(method = "lm", formula = y ~ x, se = FALSE) +
  theme_minimal() + ggtitle("Kilometerstand") + xlab("Kilometerstand") +
  ylab("Verkaufspreis in 1000 Euro")
# Gemeinsame Visualisierung:
ggarrange(plot1, plot2)
```



- ⇒ Sowohl beim Alter als auch beim Kilometerstand erscheint die Annahme eines linearen Zusammenhangs mit dem Verkaufspreis eines Autos als fragwürdig.
- ⇒ Vor allem beim Kilometerstand fällt auf, dass im Bereich zwischen 50.000 und 150.000 km ein deutlich stärkerer negativer Trend vorliegt als im Bereich zwischen 150.000 und 250.000 km.
- ⇒ Solche nicht-linearen Zusammenhänge können in einem Regressionsmodell beispielsweise durch die Aufnahme von polynomialen Termen berücksichtigt werden.

- (b) Fitten Sie ein lineares Modell in **R**, das für Alter und Kilometerstand jeweils polynomialen Terme bis zum vierten Grad inkludiert, d.h. $\text{alter}^2, \text{alter}^3, \text{alter}^4, \text{kilstand}^2, \text{kilstand}^3$ und kilstand^4 . Betrachten Sie den Output der Funktionen `summary()` und `anova()` für das Modell. Was fällt dabei auf?

Lösung:

- Modellgleichung für ein Regressionsmodell mit Polynomen vierten Grades:

$$\text{preis}_i = \beta_0 + \sum_{j=1}^4 \beta_{1j} \text{alter}_i^j + \sum_{j=1}^4 \beta_{2j} \text{kilstand}_i^j + \epsilon$$

- Schätzung des multiplen linearen Regressionsmodells:

```
model_full <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) +
                  I(alter^4) + I(kilstand) + I(kilstand^2) + I(kilstand^3) +
                  I(kilstand^4),
                  data = wagen)
summary(model_full)

##
## Call:
## lm(formula = preis ~ alter + I(alter^2) + I(alter^3) + I(alter^4) +
##     I(kilstand) + I(kilstand^2) + I(kilstand^3) + I(kilstand^4),
##     data = wagen)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.36540 -0.45524  0.04128  0.40960  3.00035
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.628e+01  4.486e+01   0.809   0.420
## alter        -9.993e-01  1.764e+00  -0.567   0.572
## I(alter^2)     1.291e-02  2.554e-02   0.506   0.614
## I(alter^3)    -7.896e-05  1.618e-04  -0.488   0.626
## I(alter^4)     1.856e-07  3.790e-07   0.490   0.625
## I(kilstand)    1.531e-02  3.522e-02   0.435   0.664
## I(kilstand^2) -4.885e-04  4.633e-04  -1.054   0.293
## I(kilstand^3)  2.883e-06  2.481e-06   1.162   0.247
## I(kilstand^4) -5.240e-09  4.609e-09  -1.137   0.257
##
## Residual standard error: 0.7452 on 163 degrees of freedom
## Multiple R-squared:  0.6591, Adjusted R-squared:  0.6424
## F-statistic: 39.39 on 8 and 163 DF,  p-value: < 2.2e-16
```

⇒ Im vollen Modell ergeben sich keine signifikant von 0 verschiedenen Parameterschätzer. Der Overall-F-Test zeigt jedoch das Vorliegen mindestens eines signifikanten Effektes an.

⇒ Eine mögliche Ursache für diese Beobachtung stellt die hohe Kollinearität innerhalb der polynomialen Variablen dar.

- Graphische Visualisierung der geschätzten Effekte:

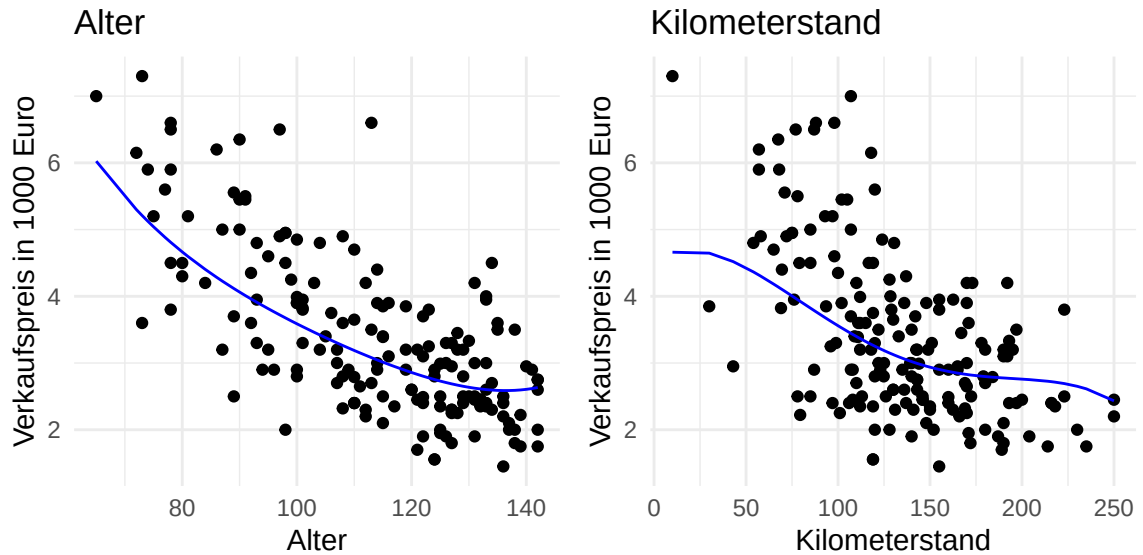
```
# Alter:
effect_alter <- wagen %>% mutate(kilstand = mean(kilstand))
effect_alter <- effect_alter %>%
  mutate(prediction_full = predict(object = model_full, newdata = effect_alter))
plot_alter <- ggplot(data = effect_alter, mapping = aes(x = alter, y = preis)) +
  geom_point() + geom_line(aes(y = prediction_full), col = "blue") +
  theme_minimal() +
  ggtitle("Alter") + xlab("Alter") + ylab("Verkaufspreis in 1000 Euro")

# Kilometerstand:
effect_km <- wagen %>% mutate(alter = mean(alter))
effect_km <- effect_km %>%
```

```

mutate(prediction_full = predict(object = model_full, newdata = effect_km))
plot_km <- ggplot(data = effect_km, mapping = aes(x = kilstand, y = preis)) +
  geom_point() + geom_line(aes(y = prediction_full), col = "blue") +
  theme_minimal() +
  ggtitle("Kilometerstand") + xlab("Kilometerstand") +
  ylab("Verkaufspreis in 1000 Euro")
# Gemeinsame Visualisierung:
ggarrange(plot_alter, plot_km)

```



- Analyse der (sequentiellen) Streuungsquadratsummen (ANOVA):

```

anova(model_full)

## Analysis of Variance Table
##
## Response: preis
##          Df Sum Sq Mean Sq  F value    Pr(>F)
## alter      1 135.435   135.435   243.8879 < 2.2e-16 ***
## I(alter^2)  1   7.949    7.949    14.3135  0.000217 ***
## I(alter^3)  1   0.016    0.016     0.0286  0.865863
## I(alter^4)  1   0.020    0.020     0.0358  0.850149
## I(kilstand) 1  27.547   27.547    49.6066 4.997e-11 ***
## I(kilstand^2) 1   3.286    3.286     5.9168  0.016079 *
## I(kilstand^3) 1   0.042    0.042     0.0748  0.784860
## I(kilstand^4) 1   0.718    0.718     1.2926  0.257242
## Residuals   163  90.516    0.555
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

⇒ Der Großteil der Streuung in der Zielgröße wird durch die linearen Terme erklärt.

⇒ Die Terme dritten und vierten Grades liefern nur sehr geringe zusätzliche Erklärung der Streuung.

- Betrachtung von Kriterien zur Modellgüte:
 - Berechnung von benötigten Kennzahlen:

```

SSE <- anova(model_full)$`Sum Sq`[9]
SSE
## [1] 90.5163
SSM <- sum(anova(model_full)$`Sum Sq`[1:8])
SSM

```

```
## [1] 175.0111
SST <- sum(anova(model_full)$`Sum Sq`)
SST <- SSM + SSE
SST
## [1] 265.5274
logLik(model_full)
## 'log Lik.' -188.8485 (df=10)
```

– Bestimmtheitsmaß R^2 :

* Berechnung:

$$R^2 = \frac{SSM}{SST} = 1 - \frac{SSE}{SST} = 1 - \frac{90.516}{265.5274} \approx 0.659$$

* Überprüfung in **R**:

```
1 - SSE / SST
## [1] 0.6591075
summary(model_full)$r.squared
## [1] 0.6591075
```

– Adjustiertes Bestimmtheitsmaß R_{adj}^2 :

* Je mehr Variablen in ein Modell inkludiert werden, desto kleiner wird R_{adj}^2 im Vergleich zu R^2 .

* Berechnung:

$$R_{adj}^2 = 1 - \frac{MSE}{MST} = 1 - \frac{\frac{SSE}{n-p'}}{\frac{SST}{n-1}} = 1 - \frac{\hat{\sigma}^2}{\frac{SST}{n-1}} \approx 0.642 < R^2$$

* Berechnung in **R**:

```
1 - summary(model_full)$sigma^2 / (SST / (nrow(wagen) - 1))
## [1] 0.6423766
summary(model_full)$adj.r.squared
## [1] 0.6423766
```

– Akaikes Informationskriterium (AIC):

* Das AIC steuert einen Kompromiss zwischen der Anpassung an die Daten und der Komplexität des Modells. Je kleiner das AIC ist, desto besser ist ein lineares Regressionsmodell.

* Berechnung:

$$AIC = -2 \cdot \loglik(model) + 2(p' + 1) = -2 \cdot 188.849 + 20 \approx 397.697$$

* Überprüfung in **R**:

```
-2 * logLik(model_full) + 2 * 10
## 'log Lik.' 397.697 (df=10)
AIC(model_full)
## [1] 397.697
```

(c) Bestimmen Sie für beide Kovariablen den optimalen Grad des Polynoms zur Modellierung des Verkaufspreises.

Lösung:

- Schätzung von linearen Regressionsmodellen mit verschiedenen Polynomgraden:

```

model_11 <- lm(formula = preis ~ alter + kilstand, data = wagen)
model_21 <- lm(formula = preis ~ alter + I(alter^2) + kilstand,
               data = wagen)
model_31 <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) + kilstand,
               data = wagen)
model_41 <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) + I(alter^4) +
               kilstand,
               data = wagen)
model_12 <- lm(formula = preis ~ alter + kilstand + I(kilstand^2),
               data = wagen)
model_22 <- lm(formula = preis ~ alter + I(alter^2) + kilstand + I(kilstand^2),
               data = wagen)
model_32 <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) + kilstand +
               I(kilstand^2),
               data = wagen)
model_42 <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) + I(alter^4) +
               kilstand + I(kilstand^2),
               data = wagen)
model_13 <- lm(formula = preis ~ alter + kilstand + I(kilstand^2) +
               I(kilstand^3),
               data = wagen)
model_23 <- lm(formula = preis ~ alter + I(alter^2) + kilstand +
               I(kilstand^2) + I(kilstand^3),
               data = wagen)
model_33 <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) + kilstand +
               I(kilstand^2) + I(kilstand^3),
               data = wagen)
model_43 <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) +
               I(alter^4) + kilstand + I(kilstand^2) + I(kilstand^3),
               data = wagen)
model_14 <- lm(formula = preis ~ alter + kilstand + I(kilstand^2) +
               I(kilstand^3) + I(kilstand^4),
               data = wagen)
model_24 <- lm(formula = preis ~ alter + I(alter^2) + kilstand +
               I(kilstand^2) + I(kilstand^3) + I(kilstand^4),
               data = wagen)
model_34 <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) + kilstand +
               I(kilstand^2) + I(kilstand^3) + I(kilstand^4),
               data = wagen)
model_44 <- lm(formula = preis ~ alter + I(alter^2) + I(alter^3) + I(alter^4) +
               kilstand + I(kilstand^2) + I(kilstand^3) + I(kilstand^4),
               data = wagen)

```

- Übersicht über die Gütekriterien für die einzelnen Modelle:

```

selection <- data.frame(matrix(data = 0, nrow = 16, ncol = 5))
colnames(selection) = c("Grad Alter", "Grad Kilometerstand", "R^2", "R_adj^2",
                       "AIC")
selection[1, ] <- c(1, 1, summary(model_11)$r.squared,
                  summary(model_11)$adj.r.squared, AIC(model_11))
selection[2, ] <- c(2, 1, summary(model_21)$r.squared,
                  summary(model_21)$adj.r.squared, AIC(model_21))
selection[3, ] <- c(3, 1, summary(model_31)$r.squared,
                  summary(model_31)$adj.r.squared, AIC(model_31))
selection[4, ] <- c(4, 1, summary(model_41)$r.squared,
                  summary(model_41)$adj.r.squared, AIC(model_41))
selection[5, ] <- c(1, 2, summary(model_12)$r.squared,

```

```

summary(model_12)$adj.r.squared, AIC(model_12))
selection[6, ] <- c(2, 2, summary(model_22)$r.squared,
summary(model_22)$adj.r.squared, AIC(model_22))
selection[7, ] <- c(3, 2, summary(model_32)$r.squared,
summary(model_32)$adj.r.squared, AIC(model_32))
selection[8, ] <- c(4, 2, summary(model_42)$r.squared,
summary(model_42)$adj.r.squared, AIC(model_42))
selection[9, ] <- c(1, 3, summary(model_13)$r.squared,
summary(model_13)$adj.r.squared, AIC(model_13))
selection[10, ] <- c(2, 3, summary(model_23)$r.squared,
summary(model_23)$adj.r.squared, AIC(model_23))
selection[11, ] <- c(3, 3, summary(model_33)$r.squared,
summary(model_33)$adj.r.squared, AIC(model_33))
selection[12, ] <- c(4, 3, summary(model_43)$r.squared,
summary(model_43)$adj.r.squared, AIC(model_43))
selection[13, ] <- c(1, 4, summary(model_14)$r.squared,
summary(model_14)$adj.r.squared, AIC(model_14))
selection[14, ] <- c(2, 4, summary(model_24)$r.squared,
summary(model_24)$adj.r.squared, AIC(model_24))
selection[15, ] <- c(3, 4, summary(model_34)$r.squared,
summary(model_34)$adj.r.squared, AIC(model_34))
selection[16, ] <- c(4, 4, summary(model_44)$r.squared,
summary(model_44)$adj.r.squared, AIC(model_44))
selection

##      Grad Alter Grad Kilometerstand      R^2  R_adj^2      AIC
## 1          1          1 0.6150780 0.6105227 406.5904
## 2          2          1 0.6433817 0.6370135 395.4540
## 3          3          1 0.6434960 0.6349569 397.3989
## 4          4          1 0.6438738 0.6331471 399.2165
## 5          1          2 0.6337535 0.6272134 400.0361
## 6          2          2 0.6558145 0.6475706 391.3505
## 7          3          2 0.6558968 0.6455323 393.3094
## 8          4          2 0.6562479 0.6437478 395.1338
## 9          1          3 0.6341828 0.6254207 401.8344
## 10         2          3 0.6559941 0.6456325 393.2607
## 11         3          3 0.6560476 0.6435402 395.2340
## 12         4          3 0.6564043 0.6417386 397.0555
## 13         1          4 0.6353701 0.6243873 403.2753
## 14         2          4 0.6586049 0.6461905 393.9504
## 15         3          4 0.6586061 0.6440344 395.9498
## 16         4          4 0.6591075 0.6423766 397.6970

```

- Auswahl des besten Modells:

```

selection[which(selection$R^2 == max(selection$R^2)), ]

##      Grad Alter Grad Kilometerstand      R^2  R_adj^2      AIC
## 16         4          4 0.6591075 0.6423766 397.697

selection[which(selection$R_adj^2 == max(selection$R_adj^2)), ]

##      Grad Alter Grad Kilometerstand      R^2  R_adj^2      AIC
## 6          2          2 0.6558145 0.6475706 391.3505

selection[which(selection$AIC == min(selection$AIC)), ]

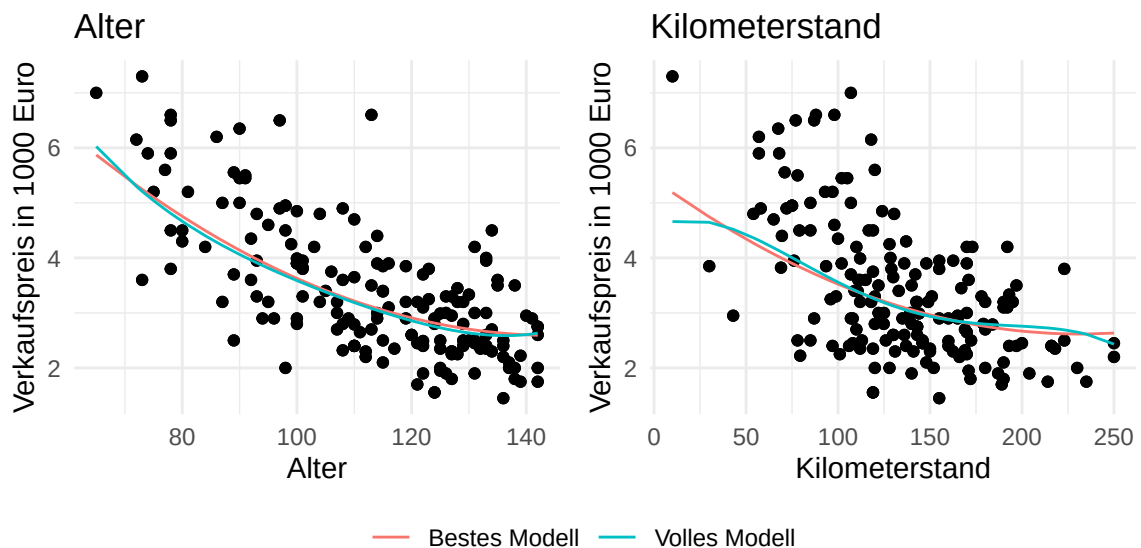
##      Grad Alter Grad Kilometerstand      R^2  R_adj^2      AIC
## 6          2          2 0.6558145 0.6475706 391.3505

```

- ⇒ Gemäß des Bestimmtheitsmaßes ist das volle Modell mit Polynomgrad 4 für beide Variablen das optimale Modell, gemäß der anderen beiden Kriterien ist das Modell mit quadratischen Termen für Alter und Kilometerstand des Autos am besten geeignet.
- ⇒ Da bei der Bestimmung von R^2_{adj} und AIC die Komplexität des Modells (Anzahl der Parameter) berücksichtigt wird, sollte die Entscheidung anhand dieser beiden Kriterien getroffen werden. Das beste Modell ist somit das Modell mit zwei quadratischen Termen.

- Graphische Visualisierung der über das beste Modell geschätzten Effekte:

```
effect_alter <- effect_alter %>%
  mutate(prediction_best = predict(object = model_22,
                                   newdata = effect_alter)) %>%
  gather(prediction_full, prediction_best, key = "model", value = "prediction")
plot_alter <- ggplot(data = effect_alter, mapping = aes(x = alter, y = preis)) +
  geom_point() + geom_line(aes(y = prediction, col = model)) + theme_minimal() +
  ggtitle("Alter") + xlab("Alter") + ylab("Verkaufspreis in 1000 Euro") +
  scale_color_discrete(name = "", labels = c("Bestes Modell", "Volles Modell"))
# Kilometerstand:
effect_km <- effect_km %>%
  mutate(prediction_best = predict(object = model_22, newdata = effect_km)) %>%
  gather(prediction_full, prediction_best, key = "model", value = "prediction")
plot_km <- ggplot(data = effect_km, mapping = aes(x = kilstand, y = preis)) +
  geom_point() + geom_line(aes(y = prediction, col = model)) + theme_minimal() +
  ggtitle("Kilometerstand") + xlab("Kilometerstand") +
  ylab("Verkaufspreis in 1000 Euro") +
  scale_color_discrete(name = "", labels = c("Bestes Modell", "Volles Modell"))
# Gemeinsame Visualisierung:
ggarrange(plotlist = list(plot_alter, plot_km), legend = "bottom",
          common.legend = TRUE)
```



- ⇒ Die geschätzten Effekte anhand der beiden betrachteten Modelle unterscheiden sich nicht allzu stark. Im Vergleich zum vollen Modell führt das Modell mit quadratischen Termen zu etwas glatteren Kurven.

- (d) Schätzen Sie nun die Effekte für das Alter und den Kilometerstand durch Regressionsplines, Truncated Power mit 3 Knoten und Polynomgrad 2. Vergleichen Sie die Modellgüte sowie die geschätzten Effekte mit dem besten Modell aus Teilaufgabe (c).

Lösung:

- Schätzung des linearen Regressionsmodells mit Regressionsplines von Grad 2:

```
model_spline <- lm(formula = preis ~ alter + I(alter^2) +
                    I((alter > 89.5) * (alter - 90)^2) +
                    I((alter > 109.5) * (alter - 110)^2) +
                    I((alter > 129.5) * (alter - 130)^2) +
                    kilstand + I(kilstand^2) +
                    I((kilstand > 99.5) * (kilstand - 100)^2) +
                    I((kilstand > 149.5) * (kilstand - 150)^2) +
                    I((kilstand > 199.5) * (kilstand - 200)^2),
                    data = wagen)
summary(model_spline)

##
## Call:
## lm(formula = preis ~ alter + I(alter^2) + I((alter > 89.5) *
##   (alter - 90)^2) + I((alter > 109.5) * (alter - 110)^2) +
##   I((alter > 129.5) * (alter - 130)^2) + kilstand + I(kilstand^2) +
##   I((kilstand > 99.5) * (kilstand - 100)^2) + I((kilstand >
##   149.5) * (kilstand - 150)^2) + I((kilstand > 199.5) * (kilstand -
##   200)^2), data = wagen)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.37830 -0.42516  0.02037  0.40735  3.05450
##
## Coefficients:
##                                Estimate Std. Error t value
## (Intercept)                   2.734e+01  1.220e+01   2.240
## alter                       -4.532e-01  2.939e-01  -1.542
## I(alter^2)                     2.351e-03  1.741e-03   1.351
## I((alter > 89.5) * (alter - 90)^2) -3.052e-03  2.522e-03  -1.210
## I((alter > 109.5) * (alter - 110)^2)  2.505e-03  1.780e-03   1.408
## I((alter > 129.5) * (alter - 130)^2) -5.020e-03  4.196e-03  -1.196
## kilstand                      1.056e-03  2.051e-02   0.051
## I(kilstand^2)                 -1.168e-04  1.265e-04  -0.923
## I((kilstand > 99.5) * (kilstand - 100)^2)  3.122e-04  2.227e-04   1.402
## I((kilstand > 149.5) * (kilstand - 150)^2) -2.300e-04  2.565e-04  -0.897
## I((kilstand > 199.5) * (kilstand - 200)^2)  8.838e-05  5.003e-04   0.177
##                                Pr(>|t|)
## (Intercept)                   0.0264 *
## alter                         0.1251
## I(alter^2)                     0.1786
## I((alter > 89.5) * (alter - 90)^2)  0.2279
## I((alter > 109.5) * (alter - 110)^2)  0.1612
## I((alter > 129.5) * (alter - 130)^2)  0.2334
## kilstand                      0.9590
## I(kilstand^2)                 0.3575
## I((kilstand > 99.5) * (kilstand - 100)^2)  0.1629
## I((kilstand > 149.5) * (kilstand - 150)^2)  0.3712
## I((kilstand > 199.5) * (kilstand - 200)^2)  0.8600
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7437 on 161 degrees of freedom
## Multiple R-squared:  0.6646, Adjusted R-squared:  0.6438
## F-statistic: 31.91 on 10 and 161 DF, p-value: < 2.2e-16
```

- Betrachtung der Modellgütekriterien:

```
# Adjustiertes Bestimmtheitsmaß:
summary(model_spline)$adj.r.squared

## [1] 0.643808

# AIC:
AIC(model_spline)

## [1] 398.8837
```

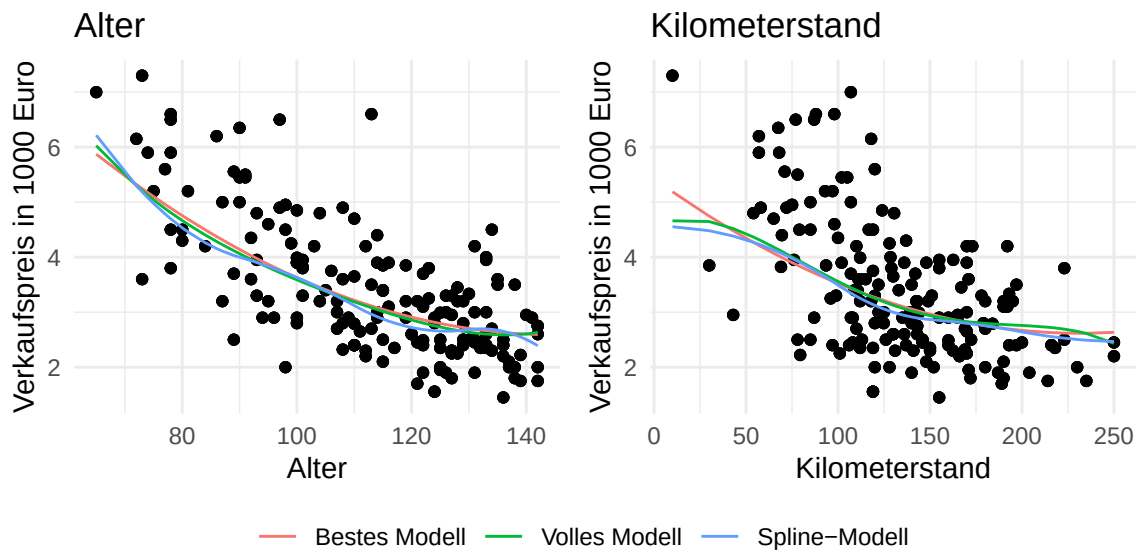
⇒ Das Modell mit Regressionssplines schneidet bezüglich beider Kriterien leicht schwächer ab als das beste Modell aus Teilaufgabe (c).

- Graphischer Vergleich der geschätzten Effekte aller drei betrachteten Modelle:

```
# Alter:
effect_alter <- wagen %>% mutate(kilstand = mean(kilstand))
effect_alter <- effect_alter %>%
  mutate(prediction_full = predict(object = model_full, newdata = effect_alter),
         prediction_best = predict(object = model_22, newdata = effect_alter),
         prediction_spline = predict(object = model_spline,
                                   newdata = effect_alter)) %>%
  gather(prediction_full, prediction_best, prediction_spline,
         key = "model", value = "prediction")
plot_alter <- ggplot(data = effect_alter, mapping = aes(x = alter, y = preis)) +
  geom_point() + geom_line(aes(y = prediction, col = model)) + theme_minimal() +
  ggtitle("Alter") + xlab("Alter") + ylab("Verkaufspreis in 1000 Euro") +
  scale_color_discrete(name = "",
                     labels = c("Bestes Modell", "Volles Modell",
                               "Spline-Modell"))

# Kilometerstand:
effect_km <- wagen %>% mutate(alter = mean(alter))
effect_km <- effect_km %>%
  mutate(prediction_full = predict(object = model_full, newdata = effect_km),
         prediction_best = predict(object = model_22, newdata = effect_km),
         prediction_spline = predict(object = model_spline,
                                   newdata = effect_km)) %>%
  gather(prediction_full, prediction_best, prediction_spline,
         key = "model", value = "prediction")
plot_km <- ggplot(data = effect_km, mapping = aes(x = kilstand, y = preis)) +
  geom_point() + geom_line(aes(y = prediction, col = model)) + theme_minimal() +
  ggtitle("Kilometerstand") + xlab("Kilometerstand") +
  ylab("Verkaufspreis in 1000 Euro") +
  scale_color_discrete(name = "",
                     labels = c("Bestes Modell", "Volles Modell",
                               "Spline-Modell"))

# Gemeinsame Visualisierung:
ggarrange(plotlist = list(plot_alter, plot_km), legend = "bottom",
          common.legend = TRUE)
```



- ⇒ Auch das Modell mit Regressionssplines führt zu relativ ähnlichen geschätzten Effekte für Alter und Kilometerstand eines Autos wie die anderen beiden Modelle.
- ⇒ Die geschätzten Effektkurven sind jeweils etwas rauer als beim optimalen Modell.

- Anmerkungen:

- Auch wenn in diesem Datenbeispiel die Anpassung des Modells mit Regressionssplines etwas schwächer ist als diejenige des besten polynomialen Modell sind spline-basierte Ansätze in vielen Situation besser geeignet, da sie deutlich mehr Flexibilität bei der Schätzung von nicht-linearen Effekten ermöglichen.
- In dieser Aufgabe wurde eine geringe Anzahl an Knoten vorab festgelegt, eine Verbesserung des Modells wäre beispielsweise durch die Berücksichtigung einer größeren Knotenanzahl möglich.
- Die Thematik der spline-basierten Regression wird in der Lehrveranstaltung “Generalisierte Regression” ausführlich behandelt.

Aufgabe 12: Modelldiagnose

Betrachten Sie das multiple lineare Regressionsmodell

$$Y_i = \beta_0 + \beta_1 \cdot x_{1i} + \beta_2 \cdot x_{2i} + \epsilon_i, \quad i = 1, \dots, n, \quad (1)$$

mit $x_{1i} = i, x_{2i} = i \bmod 2, \epsilon_i \sim N(\mu = 0, \sigma^2 = \frac{1}{3})$ unabhängig, $\beta_0 = 7, \beta_1 = 0.01$ und $\beta_2 = 1$.

- (a) Simulieren Sie drei Datensätze gemäß dieses Regressionsmodells mit $n = 10, n = 100$ und $n = 1000$ Beobachtungen. Berechnen Sie für jeden dieser Datensätze die Parameterschätzer für $\beta_0, \beta_1, \beta_2$ und σ^2 und diskutieren Sie die Ergebnisse.

Lösung:

- Vorbereitung der Simulationen:

```
# Definition von Modellparametern:
beta0 <- 7
beta1 <- 0.01
beta2 <- 1
# Funktion zur Simulation von Datensätzen mit n Beobachtungen anhand des
# gegebenen Regressionsmodells:
simulation <- function(n) {
  eps <- rnorm(n = n, mean = 0, sd = sqrt(1/3))
  x1 <- seq(1:n)
  x2 <- x1 %% 2
  y <- beta0 + beta1*x1 + beta2*x2 + eps
  data <- data.frame(eps = eps, x1 = x1, x2 = x2, y = y)
  return(data)
}
```

- Durchführung der Simulationen:

```
set.seed(1)
sim10 <- simulation(10)
head(sim10)

##           eps x1 x2          y
## 1 -0.3616833  1  1  7.648317
## 2  0.1060265  2  0  7.126027
## 3 -0.4824504  3  1  7.547550
## 4  0.9210358  4  0  7.961036
## 5  0.1902414  5  1  8.240241
## 6 -0.4736976  6  0  6.586302

sim100 <- simulation(100)
head(sim100)

##           eps x1 x2          y
## 1  0.87282726  1  1  8.882827
## 2  0.22507610  2  0  7.245076
## 3 -0.35867342  3  1  7.671327
## 4 -1.27865758  4  0  5.761342
## 5  0.64947917  5  1  8.699479
## 6 -0.02594243  6  0  7.034058

sim1000 <- simulation(1000)
head(sim1000)

##           eps x1 x2          y
## 1 -0.3670426  1  1  7.642957
```

```
## 2 -0.2665307 2 0 6.753469
## 3 0.8269285 3 1 8.856929
## 4 -0.3756797 4 0 6.664320
## 5 -0.1197313 5 1 7.930269
## 6 -0.2267878 6 0 6.833212
```

⇒ x1 stellt eine metrische Variable dar, x2 eine binäre Variable.

- Schätzung von linearen Regressionsmodellen für alle simulierten Datensätze:

```
lm10 <- lm(formula = y ~ x1 + x2, data = sim10)
summary(lm10)

##
## Call:
## lm(formula = y ~ x1 + x2, data = sim10)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.6344 -0.3760  0.1264  0.2261  0.8150
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.99677    0.39698   17.625 4.66e-07 ***
## x1           0.03732    0.05505    0.678   0.52
## x2           0.85864    0.31624    2.715   0.03 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4924 on 7 degrees of freedom
## Multiple R-squared:  0.5144, Adjusted R-squared:  0.3757
## F-statistic: 3.708 on 2 and 7 DF, p-value: 0.07979

lm100 <- lm(formula = y ~ x1 + x2, data = sim100)
summary(lm100)

##
## Call:
## lm(formula = y ~ x1 + x2, data = sim100)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.29600 -0.37568 -0.02584  0.32416  1.28473
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.014616    0.120502  58.211 < 2e-16 ***
## x1           0.010683    0.001845   5.791 8.64e-08 ***
## x2           1.052411    0.106501   9.882 2.41e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5324 on 97 degrees of freedom
## Multiple R-squared:  0.5713, Adjusted R-squared:  0.5624
## F-statistic: 64.62 on 2 and 97 DF, p-value: < 2.2e-16

lm1000 <- lm(formula = y ~ x1 + x2, data = sim1000)
summary(lm1000)
```

```
##
## Call:
## lm(formula = y ~ x1 + x2, data = sim1000)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.72393 -0.40679 -0.00712  0.41073  2.22057
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.985e+00  4.275e-02  163.42  <2e-16 ***
## x1           1.001e-02  6.614e-05  151.27  <2e-16 ***
## x2           9.918e-01  3.819e-02   25.97  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6038 on 997 degrees of freedom
## Multiple R-squared:  0.9594, Adjusted R-squared:  0.9593
## F-statistic: 1.177e+04 on 2 and 997 DF,  p-value: < 2.2e-16
```

- Vergleich der Parameterschätzer und ihrer Konfidenzintervalle:

```
# Parameterschätzer:
coef(lm10)

## (Intercept)          x1          x2
##  6.99676807  0.03731601  0.85864240

coef(lm100)

## (Intercept)          x1          x2
##  7.01461644  0.01068264  1.05241111

coef(lm1000)

## (Intercept)          x1          x2
##  6.98538766  0.01000549  0.99179486

# Konfidenzintervalle:
confint(lm10)

##              2.5 %      97.5 %
## (Intercept)  6.05806880  7.9354674
## x1          -0.09285816  0.1674902
## x2           0.11084873  1.6064361

confint(lm100)

##              2.5 %      97.5 %
## (Intercept)  6.775452432  7.25378045
## x1           0.007021344  0.01434393
## x2           0.841036818  1.26378540

confint(lm1000)

##              2.5 %      97.5 %
## (Intercept)  6.901504992  7.06927033
## x1           0.009875695  0.01013528
## x2           0.916857858  1.06673186
```

⇒ Die geschätzten Parameter liegen mit zunehmender Stichprobengröße immer näher am

tatsächlichen Wert und weisen kleinere Standardfehler und somit auch engere Konfidenzintervalle auf.

- Vergleich der geschätzten Residualstreuungen:

```
abs((summary(lm10)$sigma)^2 - 1/3)
## [1] 0.09088716

abs((summary(lm100)$sigma)^2 - 1/3)
## [1] 0.04985875

abs((summary(lm1000)$sigma)^2 - 1/3)
## [1] 0.03123592
```

⇒ Die geschätzte Varianz der Residuen $\hat{\sigma}^2$ nähert sich mit zunehmender Stichprobengröße ebenfalls immer mehr dem wahren Wert $\frac{1}{3}$ an.

- (b) Beschreiben Sie gängige Diagnosegraphiken für lineare Regressionsmodelle und erstellen Sie diese für die in Teilaufgabe (a) geschätzten Modelle.

Lösung:

- Standarddiagnosegraphiken in **R**:

– Gefittete Werte vs. Residuen:

* Darstellung: $\hat{Y}_i$ vs. $\hat{\epsilon}_i$

* Diagnostik:

· Linearität: Die Residuen sollten um die Nulllinie streuen.

· Homoskedastizität ($Var(Y_i) = \sigma^2$ für alle i): Die Residuen sollten über alle gefitteten Werte gleichmäßig streuen.

· Ausreißer

⇒ Alternativ zu dieser Darstellung können die gefitteten Werte $\hat{Y}_i$ auch gegen die beobachteten Werte Y_i der Zielgröße geplottet werden. Die beobachteten Werte sollten bei dieser Darstellung gleichmäßig um die Winkelhalbierende streuen.

– Normal-Quantil-Plot:

* Darstellung:

· Theoretische Quantile der $N(0,1)$ -Verteilung vs. Standardisierte Residuen

$$r_i = \frac{\hat{\epsilon}_i}{\hat{\sigma}\sqrt{q_{ii}}}$$

· q_{ii} bezeichnet das i -te Diagonalelement der Residualmatrix Q .

· Bei den standardisierten Residuen wird berücksichtigt, dass die Varianz der Residuen im Zentrum größer ist als an den Rändern.

* Diagnostik:

Normalverteilungsannahme: Die Punkte sollten in etwa eine Gerade bilden.

– Scale-Location-Plot:

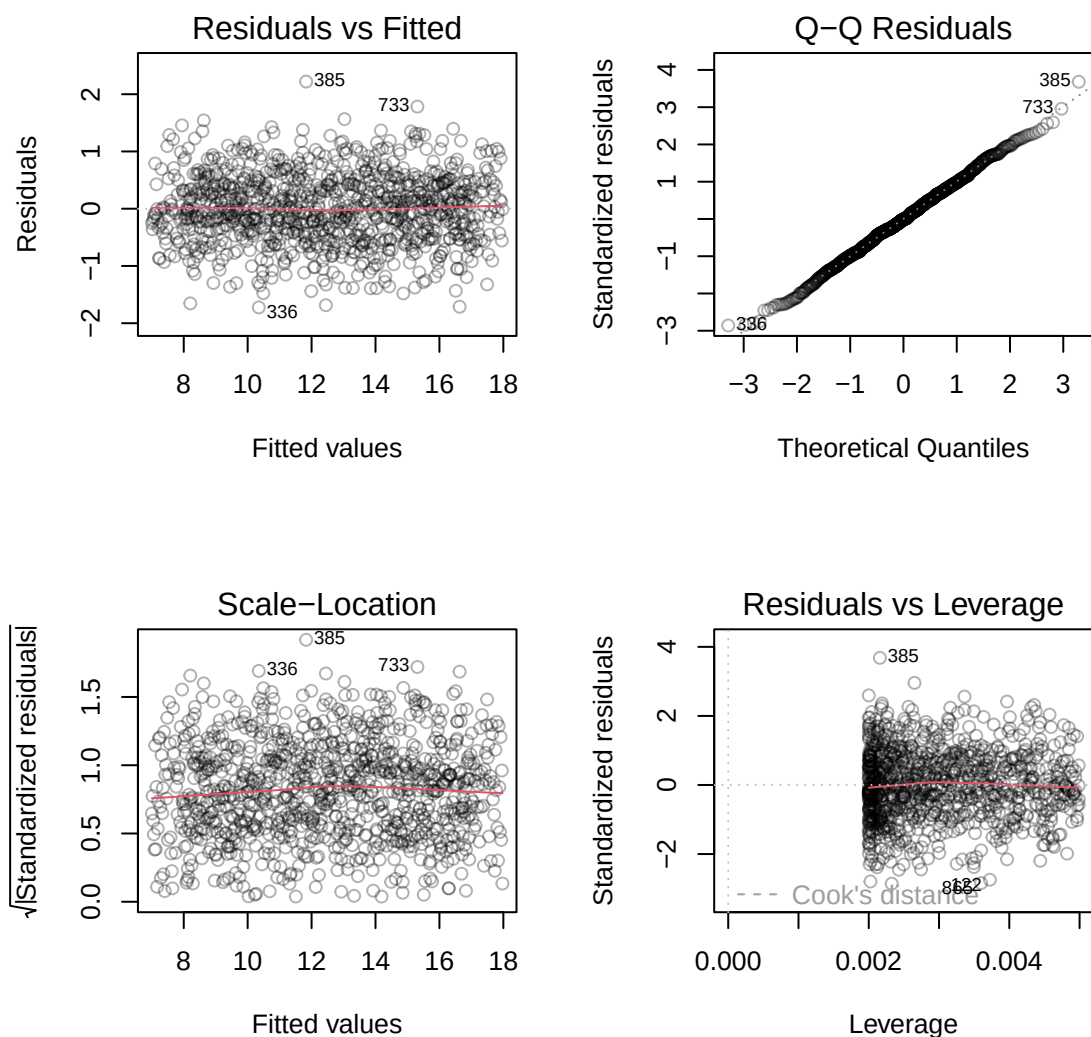
* Darstellung: $\hat{Y}_i$ vs $\sqrt{|r_i|}$

* Diagnostik:

· Homoskedastizität: Die Punkte sollten gleichmäßig entlang den Prädiktoren streuen (bestenfalls horizontale rote Linie).

- Ausreißer
- Residuals vs. Leverage:
 - * Darstellung:
 - h_{ii} vs r_i
 - Der Leverage bzw. Hebelwert $h_{ii} = x_i^\top (X^\top X)^{-1} x_i$ mit $\frac{1}{n} \leq h_{ii} \leq 1$ stellt eine Maßzahl für die Hebelwirkung der Beobachtung i auf die Schätzung des Regressionsmodells dar. Äußerst einflussreiche Beobachtungen weichen stark vom Normalwert $h_{ii} = \frac{p'}{n}$ nach oben ab.
 - Cook's Distance $D_i = \frac{r_i^2}{p'} \frac{h_{ii}}{1-h_{ii}}$ stellt eine Maßzahl zur Identifikation einflussreicher Beobachtungen mit hohen Residuen dar und lässt sich direkt aus dem Leverage berechnen.
 - * Diagnostik:
 - Einflussreiche Datenpunkte (Punkte mit Cook's Distance > 0.5)
 - Ausreißer
- Modelldiagnostik für Modell mit 1000 Beobachtungen:

```
par(mfrow= c(2, 2))
plot(x = lm1000, col = rgb(0, 0, 0, alpha = 0.3))
```



⇒ Keine der vier Diagnosegraphiken weist besondere Auffälligkeiten, es gibt also keine Hinweise auf Verletzungen der Modellannahme.

⇒ Dies lässt sich auch dadurch erklären, dass die Datenpunkte direkt aus einem linearen Regressionsmodell simuliert wurden.

- Vergleich von Diagnosegraphiken für alle drei geschätzten Modelle:

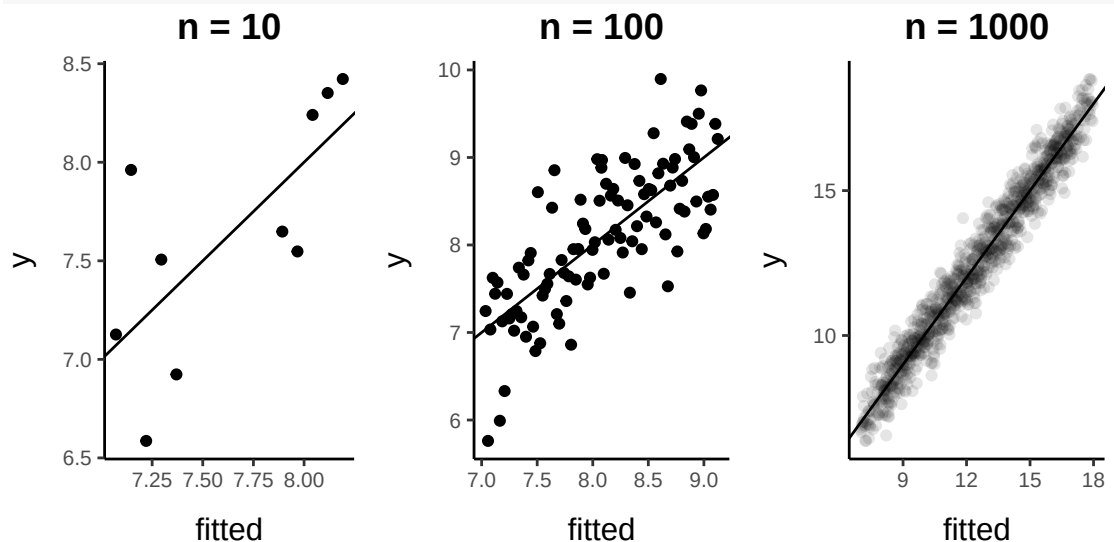
- Hinzufügen von gefitteten Werten und geschätzten Residuen zu den Datensätzen:

```
sim10 <- sim10 %>% mutate(fitted = fitted(lm10), residuals = residuals(lm10))
sim100 <- sim100 %>%
  mutate(fitted = fitted(lm100), residuals = residuals(lm100))
sim1000 <- sim1000 %>%
  mutate(fitted = fitted(lm1000), residuals = residuals(lm1000))
```

- Gefittete Werte vs. beobachtete Werte:

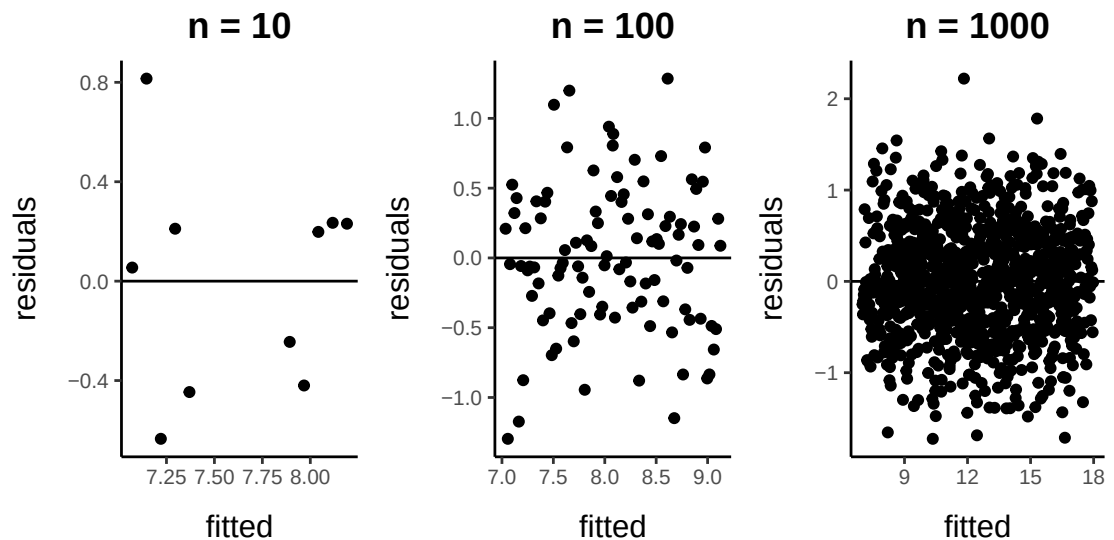
```
# install.packages("qqplotr")
library(qqplotr)

plot10 <- ggplot(data = sim10, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 100")
plot1000 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = y)) +
  geom_point(alpha = 0.1) + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



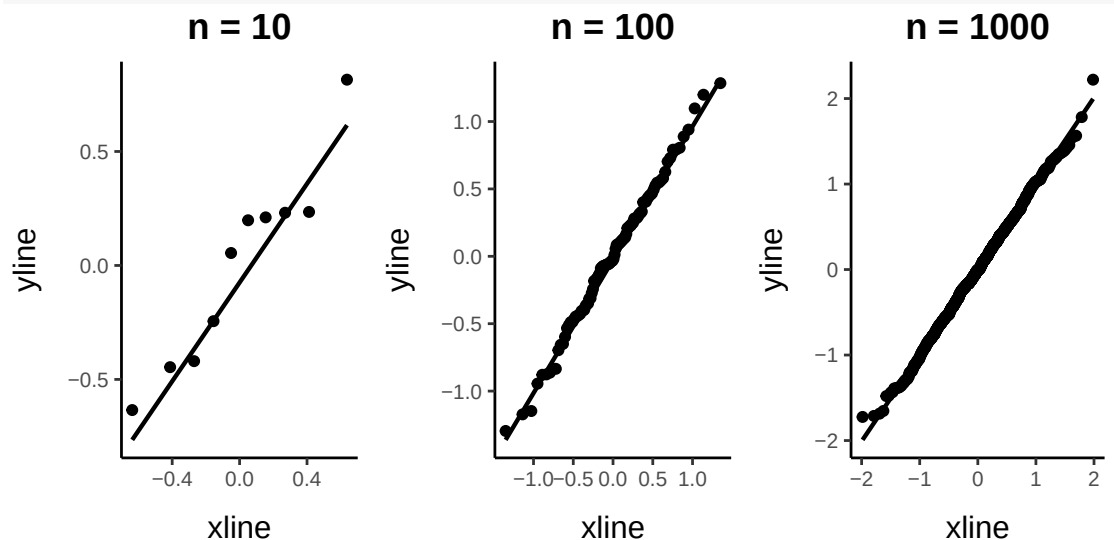
- Gefittete Werte vs. Residuen:

```
library(qqplotr)
plot10 <- ggplot(data = sim10, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 100")
plot1000 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



– Normal-Quantil-Plots:

```
library(qqplotr)
plot10 <- ggplot(data = sim10, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 100")
plot1000 <- ggplot(data = sim1000, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



– Zusammenfassung:

- * Bei allen drei Modellen sind keine Verletzungen der Modellannahmen erkennbar.
- * Je größer die Anzahl der Beobachtungen im verwendeten Datensatz ist, desto gleichmäßiger streuen die Punkte um die Nulllinie und um die Winkelhalbierende und desto eher formen die Punkte eine Gerade im Normal-Quantil-Plot.

(c) Wie verändern sich die Modellschätzungen, wenn der Einfluss von x_1 bzw. x_2 ignoriert wird? Erstellen Sie für beide Fälle erneut geeignete Diagnosegraphiken und beurteilen Sie diese.

Lösung:

- Modelle ohne x_1 :
 - Schätzung der Regressionsmodelle:

```
lm10_ohnex1 <- lm(formula = y ~ x2, data = sim10)
lm100_ohnex1 <- lm(formula = y ~ x2, data = sim100)
lm1000_ohnex1 <- lm(formula = y ~ x2, data = sim1000)
```

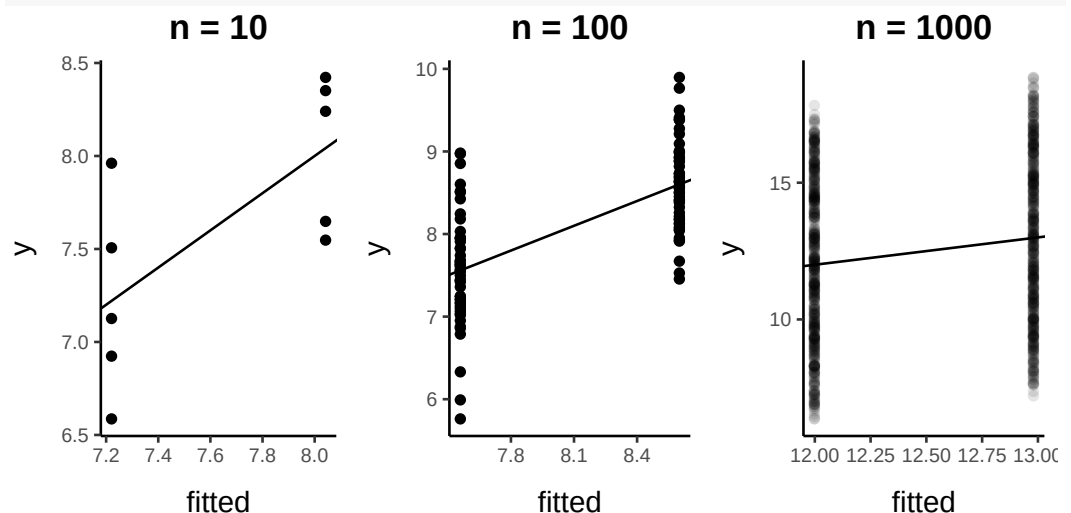
– Betrachtung von Diagnosegraphiken:

* Generierung der Datensätze für Graphiken:

```
sim10 <- sim10 %>% mutate(fitted = fitted(lm10_ohnex1),
                          residuals = residuals(lm10_ohnex1))
sim100 <- sim100 %>%
  mutate(fitted = fitted(lm100_ohnex1), residuals = residuals(lm100_ohnex1))
sim1000 <- sim1000 %>%
  mutate(fitted = fitted(lm1000_ohnex1), residuals = residuals(lm1000_ohnex1))
```

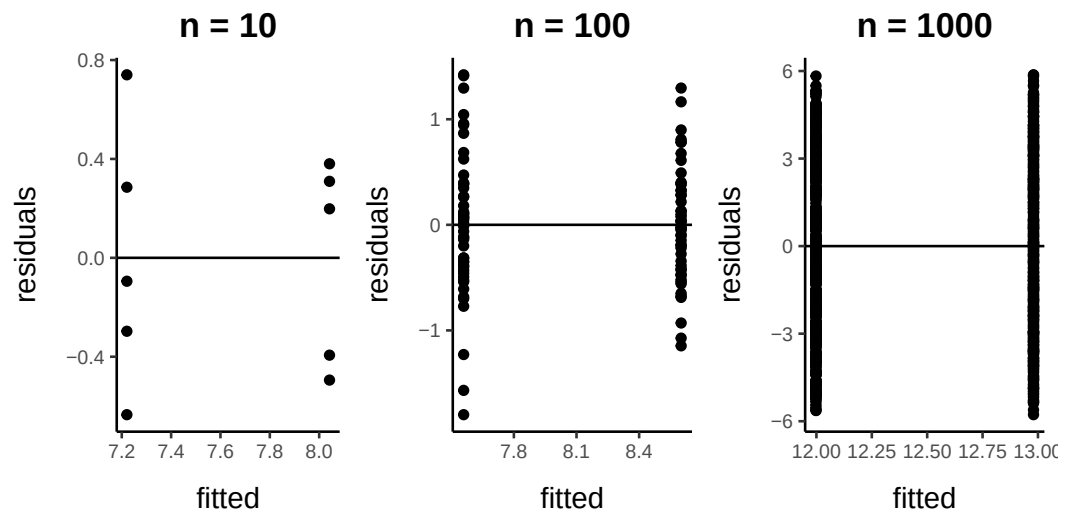
* Gefittete Werte vs. beobachtete Werte:

```
plot10 <- ggplot(data = sim10, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 100")
plot1000 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = y)) +
  geom_point(alpha = 0.1) + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



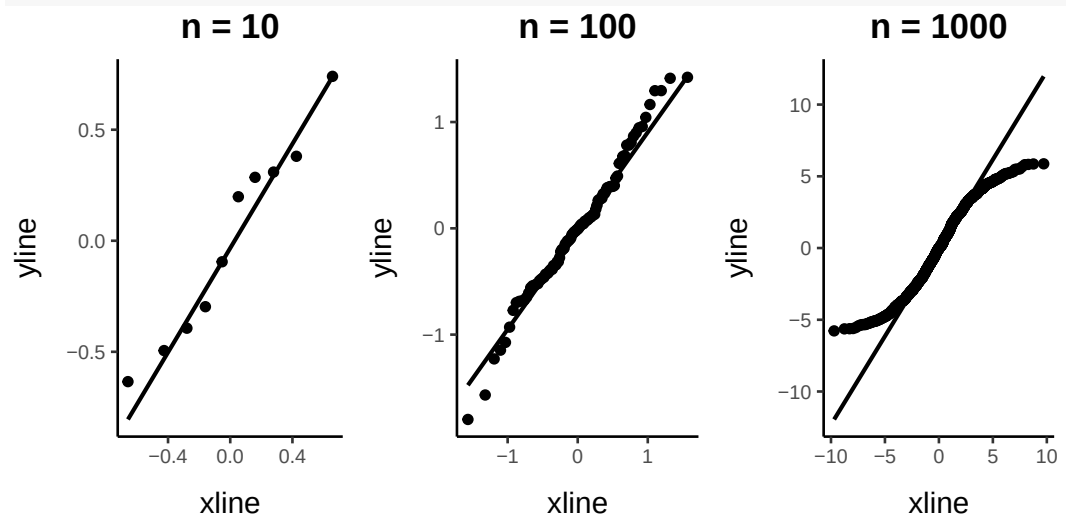
* Gefittete Werte vs. Residuen:

```
library(qqplotr)
plot10 <- ggplot(data = sim10, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 100")
plot1000 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



* Normal-Quantil-Plots:

```
plot10 <- ggplot(data = sim10, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 100")
plot1000 <- ggplot(data = sim1000, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



– Vergleich der geschätzten Parameter mit vollen Modell (nur für 1000 Beobachtungen):

```
summary(lm1000)
##
## Call:
## lm(formula = y ~ x1 + x2, data = sim1000)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.72393 -0.40679 -0.00712  0.41073  2.22057
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.985e+00  4.275e-02  163.42  <2e-16 ***
## x1           1.001e-02  6.614e-05  151.27  <2e-16 ***
## x2           9.918e-01  3.819e-02   25.97  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 0.6038 on 997 degrees of freedom
## Multiple R-squared: 0.9594, Adjusted R-squared: 0.9593
## F-statistic: 1.177e+04 on 2 and 997 DF, p-value: < 2.2e-16
summary(lm1000_ohnex1)
##
## Call:
## lm(formula = y ~ x2, data = sim1000)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.7752 -2.4419 -0.0357  2.4604  5.8701
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  11.9981     0.1321   90.836 < 2e-16 ***
## x2           0.9818     0.1868    5.256  1.8e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.954 on 998 degrees of freedom
## Multiple R-squared: 0.02693, Adjusted R-squared: 0.02596
## F-statistic: 27.62 on 1 and 998 DF, p-value: 1.801e-07
```

– Beurteilung:

- * Da die Regressionsmodelle mit einer einzigen binären Einflussvariable geschätzt wurden, sind Diagnosegraphiken mit gefitteten Werten wenig aussagekräftig.
- * Die Normal-Quantil-Plots zeigen starke Abweichungen von der Normalverteilungsannahme der Residuen.
- * Die geschätzten Koeffizienten sind verzerrt und weisen viel größere Standardfehler auf als beim vollen Modell.
- * Die geschätzte Residualstreuung ist deutlich größer als beim vollen Modell.

• Modelle ohne x_2 :

– Schätzung der Regressionsmodelle:

```
lm10_ohnex2 <- lm(formula = y ~ x1, data = sim10)
lm100_ohnex2 <- lm(formula = y ~ x1, data = sim100)
lm1000_ohnex2 <- lm(formula = y ~ x1, data = sim1000)
```

– Betrachtung von Diagnosegraphiken:

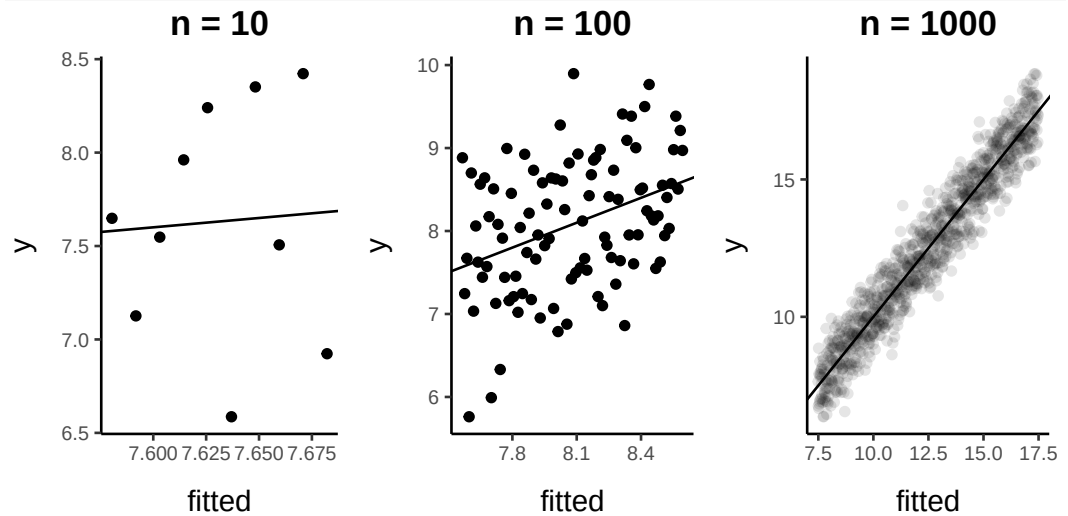
- * Generierung der Datensätze für Graphiken:

```
sim10 <- sim10 %>% mutate(fitted = fitted(lm10_ohnex2),
                          residuals = residuals(lm10_ohnex2))
sim100 <- sim100 %>%
  mutate(fitted = fitted(lm100_ohnex2), residuals = residuals(lm100_ohnex2))
sim1000 <- sim1000 %>%
  mutate(fitted = fitted(lm1000_ohnex2), residuals = residuals(lm1000_ohnex2))
```

- * Gefittete Werte vs. beobachtete Werte:

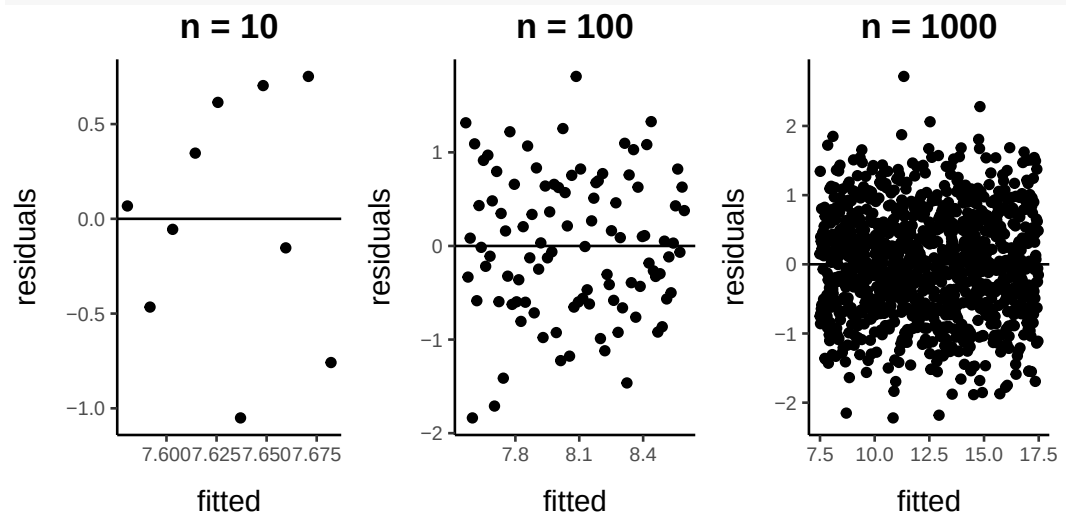
```
plot10 <- ggplot(data = sim10, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 100")
```

```
plot1000 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = y)) +
  geom_point(alpha = 0.1) + geom_abline(intercept = 0, slope = 1) + theme +
  ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



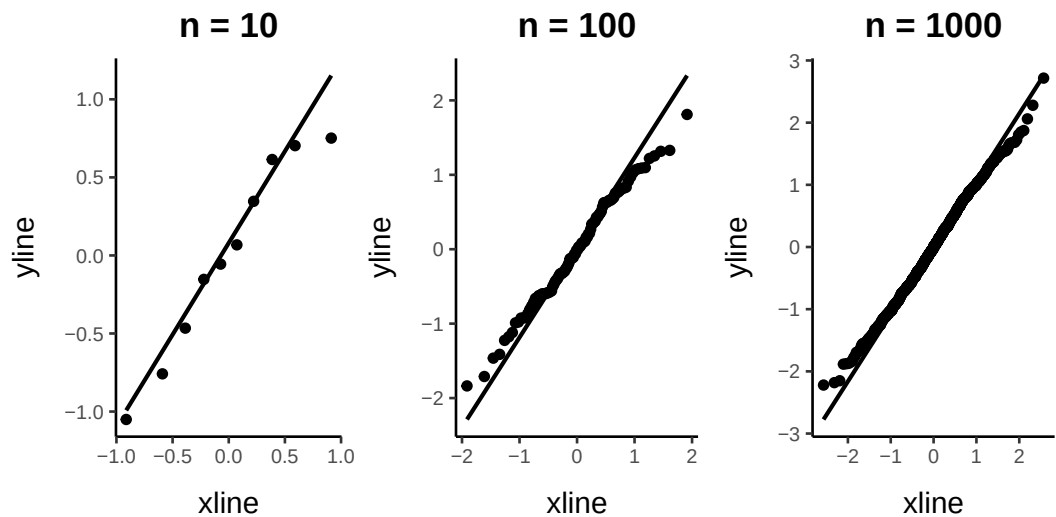
* Gefittete Werte vs. Residuen:

```
plot10 <- ggplot(data = sim10, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 100")
plot1000 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0) + theme + ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



* Normal-Quantil-Plots:

```
plot10 <- ggplot(data = sim10, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 10")
plot100 <- ggplot(data = sim100, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 100")
plot1000 <- ggplot(data = sim1000, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line() + theme + ggtitle("n = 1000")
ggarrange(plot10, plot100, plot1000, ncol = 3)
```



– Vergleich der geschätzten Parameter mit vollen Modell (nur für 1000 Beobachtungen):

```
summary(lm1000)
##
## Call:
## lm(formula = y ~ x1 + x2, data = sim1000)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.72393 -0.40679 -0.00712  0.41073  2.22057
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.985e+00  4.275e-02  163.42  <2e-16 ***
## x1           1.001e-02  6.614e-05  151.27  <2e-16 ***
## x2           9.918e-01  3.819e-02   25.97  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6038 on 997 degrees of freedom
## Multiple R-squared:  0.9594, Adjusted R-squared:  0.9593
## F-statistic: 1.177e+04 on 2 and 997 DF,  p-value: < 2.2e-16

summary(lm1000_ohnex2)
##
## Call:
## lm(formula = y ~ x1, data = sim1000)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.22032 -0.57503  0.00069  0.55870  2.71612
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.4827743  0.0494581   151.3  <2e-16 ***
## x1           0.0100025  0.0000856   116.9  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7814 on 998 degrees of freedom
## Multiple R-squared:  0.9319, Adjusted R-squared:  0.9318
## F-statistic: 1.365e+04 on 1 and 998 DF,  p-value: < 2.2e-16
```

- Beurteilung:
 - * Die Diagnosegraphiken weisen keine größeren Auffälligkeiten auf.
 - * Der Koeffizientenschätzer für die Variable x_1 ist kaum verzerrt.
 - * Die Standardfehler der geschätzten Parameter sind im Vergleich zum vollen Modell etwas größer.
- (d) Betrachten Sie nun verschiedene Modellabweichungen von Modell (1) mit $n = 1000$ Beobachtungen:
 - (i) Setzen Sie im obigen Datensatz für eine Beobachtung Y_i einen extremen Wert (Ausreißer) ein.

Lösung:

- Einfügen eines extremen Ausreißers in den bestehenden Datensatz:

```
yneu <- sim1000$y
yneu[10] <- 20
```

⇒ Für die zehnte Beobachtung wird ein y-Wert von 20 eingesetzt.

- Schätzung des Regressionsmodells:

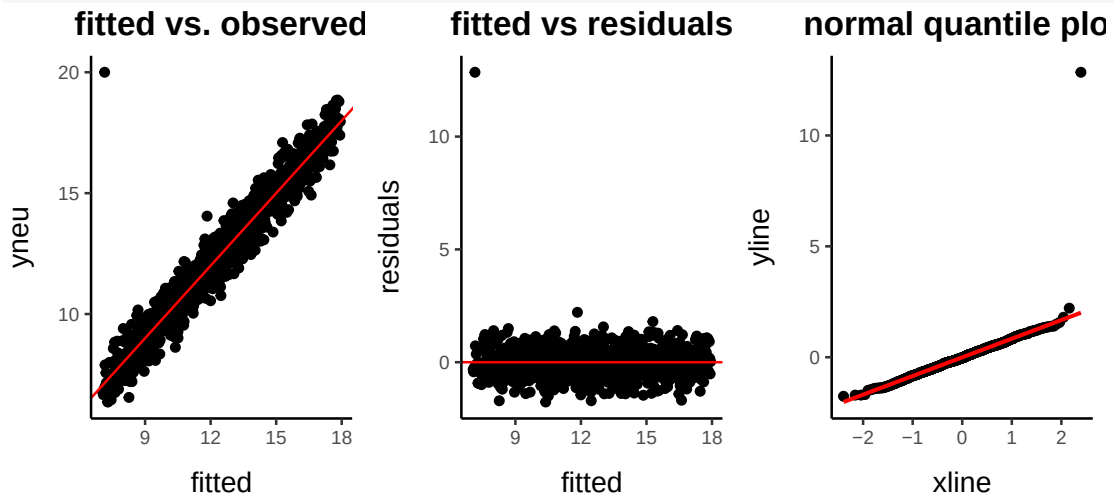
```
lm1000i <- lm(formula = yneu ~ x1 + x2, data = sim1000)
summary(lm1000i)

##
## Call:
## lm(formula = yneu ~ x1 + x2, data = sim1000)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.7626 -0.4189 -0.0280  0.4061 12.8509
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.050e+00  5.159e-02  136.65  <2e-16 ***
## x1           9.929e-03  7.983e-05  124.38  <2e-16 ***
## x2           9.657e-01  4.609e-02   20.95  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7287 on 997 degrees of freedom
## Multiple R-squared:  0.941, Adjusted R-squared:  0.9409
## F-statistic: 7950 on 2 and 997 DF, p-value: < 2.2e-16
```

- Betrachtung von Diagnosegraphiken:

```
# Generierung des benötigten Datensatzes:
sim1000 <- sim1000 %>%
  mutate(fitted = fitted(lm1000i), residuals = residuals(lm1000i))
# Gefittete Wertes vs. beobachtete Werte:
plot1 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = yneu)) +
  geom_point() + geom_abline(intercept = 0, slope = 1, col = "red") + theme +
  ggtitle("fitted vs. observed")
# Gefittete Werte vs. Residuen:
plot2 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0, col = "red") + theme +
  ggtitle("fitted vs residuals")
# Normal-Quantil-Plot:
plot3 <- ggplot(data = sim1000, mapping = aes(sample = residuals)) +
```

```
stat_qq_point() + stat_qq_line(col = "red") + theme +
ggtitle("normal quantile plot")
# Gemeinsame Visualisierung:
ggarrange(plot1, plot2, plot3, ncol = 3)
```



– Betrachtung von Cook's Distance:

```
head(sort(x = cooks.distance(lm1000i), decreasing = TRUE))
##          10          122          385          865          944          733
## 0.512238327 0.006878216 0.006657528 0.006454511 0.005433778 0.005426621
```

– Beurteilung:

- * Da der Stichprobenumfang sehr groß ist, führt die extreme Beobachtung nur zu einer geringen Verzerrung der Parameterschätzer. Bei kleinerem n wären die Auswirkungen deutlich größer.
- * Die geänderte Beobachtung kann über alle Diagnosegraphiken klar als Ausreißer identifiziert werden.
- * Die Beobachtung kann auch als sehr einflussreicher Datenpunkt angesehen werden, sie weist deutlich die größte Cook's Distance unter allen Beobachtungen auf.

- (ii) Simulieren Sie einen neuen Datensatz gemäß Modell (1), wobei die Fehler ϵ_i nun keiner Normalverteilung, sondern einer Gleichverteilung auf dem Intervall $[-1, 1]$ folgen sollen.

Lösung:

– Simulation des neuen Datensatzes:

```
# Funktion zur Simulation des neuen Datensatzes mit gleichverteilten
# Fehlern:
simulation_unif <- function(n) {
  eps <- runif(n = n, min = -1, max = 1)
  x1 <- seq(1:n)
  x2 <- x1 %% 2
  y <- beta0 + beta1*x1 + beta2*x2 + eps
  data <- data.frame(eps = eps, x1 = x1, x2 = x2, y = y)
  return(data)
}
# Durchführung der Simulationen:
set.seed(1)
sim1000 <- simulation_unif(1000)
head(sim1000)
##          eps x1 x2          y
## 1 -0.4689827  1  1  7.541017
## 2 -0.2557522  2  0  6.764248
```

```
## 3  0.1457067  3  1 8.175707
## 4  0.8164156  4  0 7.856416
## 5 -0.5966361  5  1 7.453364
## 6  0.7967794  6  0 7.856779
```

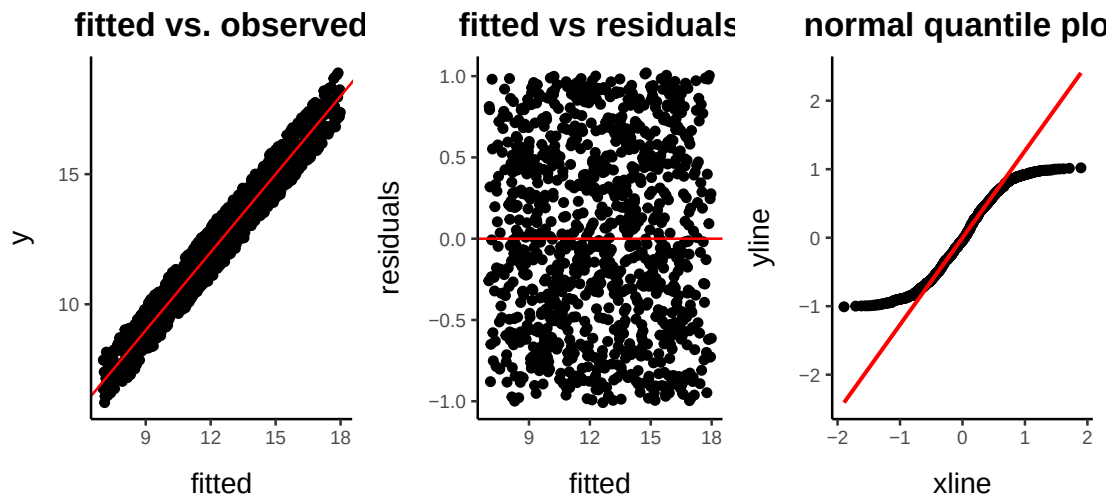
– Schätzung des Regressionsmodells:

```
lm1000ii <- lm(formula = y ~ x1 + x2, data = sim1000)
summary(lm1000ii)

##
## Call:
## lm(formula = y ~ x1 + x2, data = sim1000)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.00841 -0.49406 -0.03838  0.49261  1.02003
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.003e+00  4.086e-02  171.40  <2e-16 ***
## x1           9.969e-03  6.322e-05  157.67  <2e-16 ***
## x2           1.023e+00  3.650e-02   28.04  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5772 on 997 degrees of freedom
## Multiple R-squared:  0.9626, Adjusted R-squared:  0.9625
## F-statistic: 1.282e+04 on 2 and 997 DF,  p-value: < 2.2e-16
```

– Betrachtung von Diagnosegraphiken:

```
# Generierung des benötigten Datensatzes:
sim1000 <- sim1000 %>%
  mutate(fitted = fitted(lm1000ii), residuals = residuals(lm1000ii))
# Gefittete Wertes vs. beobachtete Werte:
plot1 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1, col = "red") + theme +
  ggtitle("fitted vs. observed")
# Gefittete Werte vs. Residuen:
plot2 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0, col = "red") + theme +
  ggtitle("fitted vs residuals")
# Normal-Quantil-Plot:
plot3 <- ggplot(data = sim1000, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line(col = "red") + theme +
  ggtitle("normal quantile plot")
# Gemeinsame Visualisierung:
ggarrange(plot1, plot2, plot3, ncol = 3)
```



– Beurteilung:

- * Der Normal-Quantil-Plot zeigt, dass die Residuen des Modells nicht normalverteilt sind.
- * Die Parameterschätzer werden dadurch nicht verzerrt, allerdings verlieren Konfidenzintervalle ihre Gültigkeit.

(iii) Simulieren Sie einen neuen Datensatz gemäß Modell (1) mit heteroskedastischen Fehlern $\epsilon_i \sim N(\mu = 0, \sigma^2 = \frac{2i}{300})$.

Lösung:

– Simulation des neuen Datensatzes:

```
# Funktion zur Simulation des neuen Datensatzes mit heteoskedastischen
# Fehlern:
simulation_het <- function(n) {
  x1 <- seq(1:n)
  eps <- rnorm(n = n, mean = 0, sd = 2 * x1 / 300)
  x2 <- x1 %%2
  y <- beta0 + beta1*x1 + beta2*x2 + eps
  data <- data.frame(eps = eps, x1 = x1, x2 = x2, y = y)
  return(data)
}

# Durchführung der Simulationen:
set.seed(1)
sim1000 <- simulation_het(1000)
head(sim1000)

##           eps x1 x2          y
## 1 -0.004176359  1  1  8.005824
## 2  0.002448578  2  0  7.022449
## 3 -0.016712572  3  1  8.013287
## 4  0.042540821  4  0  7.082541
## 5  0.010983592  5  1  8.060984
## 6 -0.032818735  6  0  7.027181
```

– Schätzung des Regressionsmodells:

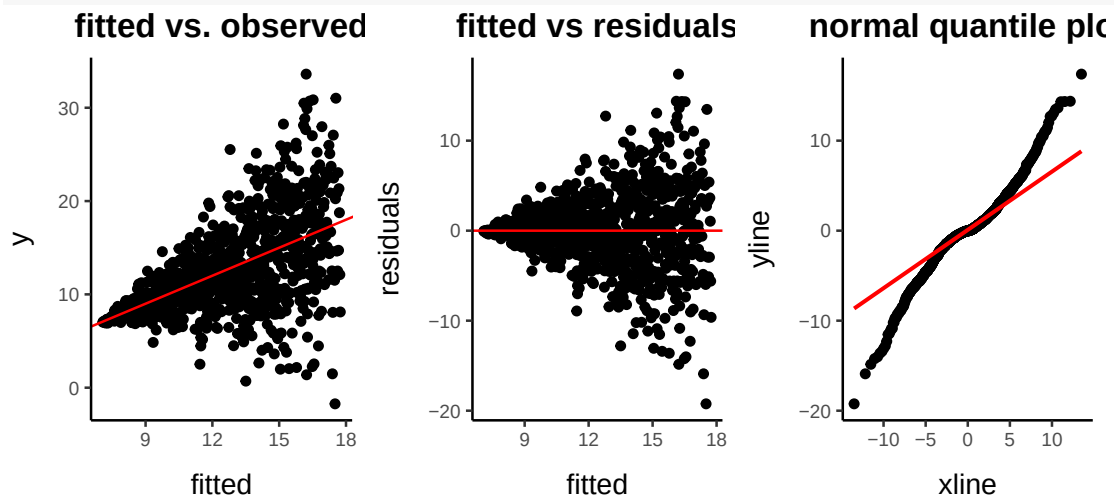
```
lm1000iii <- lm(formula = y ~ x1 + x2, data = sim1000)
summary(lm1000iii)

##
## Call:
## lm(formula = y ~ x1 + x2, data = sim1000)
##
## Residuals:
```

```
##      Min      1Q   Median      3Q      Max
## -19.2383 -1.7042 -0.0201  1.8771 17.3907
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.0646081  0.2907792  24.295  < 2e-16 ***
## x1           0.0098142  0.0004499  21.813  < 2e-16 ***
## x2           0.8746430  0.2597690   3.367  0.000789 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.107 on 997 degrees of freedom
## Multiple R-squared:  0.3281, Adjusted R-squared:  0.3268
## F-statistic: 243.4 on 2 and 997 DF, p-value: < 2.2e-16
```

– Betrachtung von Diagnosegraphiken:

```
# Generierung des benötigten Datensatzes:
sim1000 <- sim1000 %>%
  mutate(fitted = fitted(lm1000iii), residuals = residuals(lm1000iii))
# Gefittete Wertes vs. beobachtete Werte:
plot1 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1, col = "red") + theme +
  ggtitle("fitted vs. observed")
# Gefittete Werte vs. Residuen:
plot2 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = residuals)) +
  geom_point() + geom_hline(yintercept = 0, col = "red") + theme +
  ggtitle("fitted vs residuals")
# Normal-Quantil-Plot:
plot3 <- ggplot(data = sim1000, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line(col = "red") + theme +
  ggtitle("normal quantile plot")
# Gemeinsame Visualisierung:
ggarrange(plot1, plot2, plot3, ncol = 3)
```



– Beurteilung:

- * Die Diagnosegraphiken zeigen das Vorliegen von heteroskedastischen Residuen, mit zunehmenden gefitteten Werten nimmt auch die Streuung der Residuen zu.
- * Die Parameterschätzer sind auch bei Heteroskedastie weiterhin erwartungstreu, allerdings weisen die Schätzer nicht mehr die kleinstmögliche Varianz auf.
- * Auch die Normalverteilungsannahme bezüglich der Residuen ist nicht wirklich gegeben.

(iv) Simulieren Sie einen neuen Datensatz gemäß Modell (1), wobei der Term $\beta_1 \cdot x_{1i}$ in der

Modellgleichung durch $\beta_1 \cdot \frac{x_{1i}^2}{100}$ ersetzt werden soll.

Lösung:

- Simulation des neuen Datensatzes:

```
# Funktion zur Simulation des neuen Datensatzes:
simulation_nonlinear <- function(n) {
  eps <- rnorm(n = n, mean = 0, sd = sqrt(1/3))
  x1 <- seq(1:n)
  x2 <- x1 %%2
  y <- beta0 + beta1 * (x1^2 / 100) + beta2 * x2 + eps
  data <- data.frame(eps = eps, x1 = x1, x2 = x2, y = y)
  return(data)
}
# Durchführung der Simulationen:
set.seed(1)
sim1000 <- simulation_nonlinear(1000)
head(sim1000)

##           eps x1 x2           y
## 1 -0.3616833  1  1  7.638417
## 2  0.1060265  2  0  7.106427
## 3 -0.4824504  3  1  7.518450
## 4  0.9210358  4  0  7.922636
## 5  0.1902414  5  1  8.192741
## 6 -0.4736976  6  0  6.529902
```

- Schätzung des Regressionsmodells:

```
lm1000iv <- lm(formula = y ~ x1 + x2, data = sim1000)
summary(lm1000iv)

##
## Call:
## lm(formula = y ~ x1 + x2, data = sim1000)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -9.778  -6.797  -2.073   5.729  17.209
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -9.6869413   0.5315698  -18.223  <2e-16 ***
## x1           0.1000457   0.0008225  121.634  <2e-16 ***
## x2           0.9813916   0.4748805   2.067    0.039 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.509 on 997 degrees of freedom
## Multiple R-squared:  0.9369, Adjusted R-squared:  0.9368
## F-statistic: 7399 on 2 and 997 DF, p-value: < 2.2e-16
```

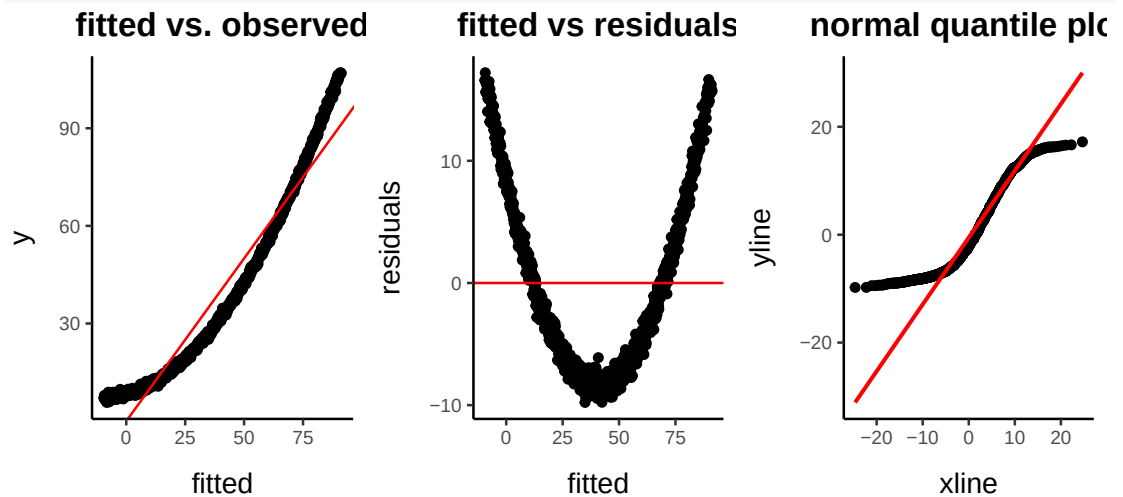
- Betrachtung von Diagnosegraphiken:

```
# Generierung des benötigten Datensatzes:
sim1000 <- sim1000 %>%
  mutate(fitted = fitted(lm1000iv), residuals = residuals(lm1000iv))
# Gefittete Wertes vs. beobachtete Werte:
plot1 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = y)) +
  geom_point() + geom_abline(intercept = 0, slope = 1, col = "red") + theme +
  ggtitle("fitted vs. observed")
# Gefittete Werte vs. Residuen:
plot2 <- ggplot(data = sim1000, mapping = aes(x = fitted, y = residuals)) +
```

```

geom_point() + geom_hline(yintercept = 0, col = "red") + theme +
ggtitle("fitted vs residuals")
# Normal-Quantil-Plot:
plot3 <- ggplot(data = sim1000, mapping = aes(sample = residuals)) +
  stat_qq_point() + stat_qq_line(col = "red") + theme +
  ggtitle("normal quantile plot")
# Gemeinsame Visualisierung:
ggarrange(plot1, plot2, plot3, ncol = 3)

```



– Beurteilung:

- * Die Diagnosegraphiken zeigen eine eindeutige Struktur in den Residuen über die gefitteten Werte, es liegt ein nichtlinearer Zusammenhang vor.
- * Das Modell mit ausschließlich linearen Effekten liefert nur eine (lineare) Näherung des wahren (quadratischen) Zusammenhangs.

Aufgabe 13: Überprüfung von Modellannahmen

Betrachten Sie das lineare Modell $Y = X\beta + \epsilon$ mit der Annahme $\epsilon \sim \mathcal{N}_n(0, \sigma^2 I_n)$ bezüglich der Verteilung der Störterme. In dieser Aufgabe sollen die in dieser Annahme enthaltenen Aspekte genauer betrachtet werden.

- (a) Beschreiben Sie mögliche Abweichungen von dieser Annahme.

Lösung:

Mögliche Abweichungen:

- Die Varianzhomogenität ist nicht erfüllt, d.h. die Störterme ϵ_i sind nicht identisch verteilt und $\text{Var}(\epsilon_i) = \sigma^2$ ($i = 1, \dots, n$) gilt nicht.
 - Die Störterme sind nicht unabhängig voneinander, d.h. $\epsilon_i \perp \epsilon_j \forall i \neq j$ gilt nicht.
⇒ Die Gültigkeit der Annahme kann durch einen ACF-Plot oder den Durbin-Watson-Test überprüft werden.
 - Die Normalverteilungsannahme gilt nicht, d.h. die Störterme ϵ_i sind nicht normalverteilt.
 - Die Zentrierung der Störterme ist nicht erfüllt, d.h. $\mathbb{E}(\epsilon_i) = 0$ ($i = 1, \dots, n$) gilt nicht.
- (b) Im Folgenden sind einige Anwendungsbeispiele für lineare Modelle skizziert. Die Abweichungen von welcher Annahme würden Sie in den untenstehenden Beispielen je primär untersuchen? Welche Diagnostik würden Sie jeweils verwenden, um die entsprechenden Annahmen zu überprüfen?

- (i) Y : Konsum in einem Quartal (in Euro)
 X : verschiedene ökonomische Größen

Lösung:

Da der Konsum nur positive Werte annimmt und möglicherweise rechtsschief verteilt ist, sollte die Normalverteilungsannahme überprüft werden.

⇒ Diagnostik: Normal-Quantil-Plot.

- (ii) Y : Gewichtsreduktion nach einer Therapie (in kg)
 X : Gruppierungsvariable (drei Kategorien), Gewicht zu Beginn, Alter

Lösung:

Personen mit höherem Gewicht können potentiell mehr abnehmen als Personen mit geringerem Gewicht. Somit könnten die Fehler für Personen mit höherem Gewicht eine größere Streuung aufweisen.

⇒ Diagnostik: Scale-Location-Plot, Residuen gegen gefittete Werte

- (iii) Y : Mathematikfähigkeit (gemessen mit Score, erhoben bei allen Schülern einer Schule)
 X : Alter, Geschlecht, IQ

Lösung:

Ein Klasseneffekt ist möglich, dann wären die einzelnen Messungen nicht unabhängig voneinander, da Messungen innerhalb einer Klasse zueinander ähnlicher sind als Messungen aus verschiedenen Klassen.

⇒ Diagnostik: Residuen gegen gefittete Werte

⇒ Mögliche Lösung: Random Intercept-Modell

- (iv) Y : Zeit bis zum Verlassen des Krankenhauses
 X : Typ der Krankheit (kategorial), Geschlecht, Alter

Lösung:

Da die Zeit nur positive Werte annehmen kann, sollte die Normalverteilungsannahme überprüft werden.

⇒ Diagnostik: Normal-Quantil-Plot

June 22, 2026

Fragen und Kommentare an laura.bader@stat.uni-muenchen.de